



Generative AI & LLM Fundamentals

Complete Instructor Guide — 4-Hour Session

LLM Lifecycle · Decoding Strategies · AWS Bedrock · Boto3 Hands-on

AWS GenAI Foundation Course | Module 1 | Session 1 | 4 Hours

Table of Contents

What you will learn and build today

Hour 1 — LLM Lifecycle Deep Dive

Topics: Data→Tokenization→Pre-training→Fine-tuning→RLHF→Eval→Deploy

Lab: GPT-2 tokenization, probabilities, temperature, fine-tuning on Colab

Hour 2 — Decoding Strategies

Topics: Temperature · Top-K · Top-P · Beam Search · Greedy · Repetition

Lab: Compare all 5 strategies side-by-side on the same prompt

Hour 3 — AWS Bedrock Ecosystem

Topics: Serverless AI · Model Catalog · Security · Guardrails · Pricing

Lab: Activate models, use Playground, compare Claude vs Llama vs Titan

Hour 4 — Hands-on Boto3 Lab

Topics: IAM · InvokeModel · Streaming · Token Counting · Production Class

Lab: Build a multi-model benchmark script with cost tracking

Hour 1

LLM Lifecycle Deep Dive

From raw text on the internet to a deployed AI assistant — every step explained

30 min Theory + 30 min Colab Lab (5 exercises)

What is an LLM?

The big picture before any technical details

The Core Idea:

An LLM is a massive mathematical model that learns to predict the next word by reading billions of sentences. Nothing more, nothing less — but from that simple task emerges deep language understanding.

The Human Analogy:

Think of a child learning Arabic. No grammar rules were memorized. They simply heard millions of sentences and their brain auto-calibrated. If someone says 'I'm going to...' the brain instantly predicts 'school' or 'work' — never 'elephant'.

■ *LLMs do exactly this — but with billions of numbers (parameters) instead of neurons.*

Key Insight

After training, the model gains an **implicit understanding** of language, science, and history — not because it was programmed with that knowledge, but because of the patterns in the text.

Scale Matters

GPT-2 (2019) → 117 Million parameters

GPT-3 (2020) → 175 Billion parameters

GPT-4 (2023) → ~1 Trillion+ parameters

Claude 3.5 → Unknown (>100B estimated)

Historical Evolution of NLP

From counting words to emergent intelligence — 4 eras

Era	Model Type	Key Models	Key Breakthrough
1990s–2010	Statistical LMs (SLMs)	N-gram, HMM	Count word co-occurrences in huge corpora
2013–2017	Neural LMs (NLMs)	Word2Vec, LSTM, GRU	Learn dense vector representations of words
2018–2019	Pre-trained LMs (PLMs)	BERT, GPT-1, ELMo	Transfer learning: pre-train then fine-tune
2020→Now	Large LMs (LLMs)	GPT-3/4, Claude, Gemini	Scale creates emergent abilities

What are 'Emergent Abilities'?

When you scale a model past ~10B parameters, capabilities appear that WEREN'T present in smaller models and weren't explicitly trained for:

- ◆ Multi-step arithmetic (GPT-3 couldn't, GPT-4 can)
- ◆ Chain-of-thought reasoning
- ◆ Code generation from natural language descriptions
- ◆ In-context learning (learn from examples in the prompt)

■ *The jump from PLMs to LLMs is not just size — it is a qualitative leap in capability.*

The LLM Lifecycle — 9 Stages

Complete map from zero to production

1	Data Collection	45TB+ of internet text, books, code, papers
2	Data Preprocessing	Dedup · Quality Filter · PII Removal · Toxic Content
3	Tokenization	Convert text to numbers using BPE / WordPiece / SentencePiece
4	Pre-training	Self-supervised Next Token Prediction on billions of tokens
5	Supervised Fine-Tuning	Transform 'text predictor' into 'instruction follower'
6	Alignment (RLHF)	Align behavior with human values using reinforcement learning
7	Evaluation	Measure on MMLU, HumanEval, TruthfulQA, GSM8K benchmarks
8	Deployment	Optimize with Quantization, KV Cache, vLLM, batching
9	Monitoring	Drift detection, hallucination tracking, feedback loop

Stage 1 & 2 — Data & Preprocessing

The quality of the data determines the quality of the model

Data Sources (GPT-3 Example):

Where the Data Comes From

Common Crawl → Petabytes (entire internet crawl)

Books / BookCorpus → Billions of words

Wikipedia (all langs) → ~21 GB clean text

GitHub / Stack Overflow → Terabytes of code

ArXiv / PubMed → Millions of academic papers

Preprocessing Steps:

- ◆ Deduplication — remove identical/near-identical text

- Same paragraph appearing 500x warps the model

- ◆ Quality Filtering — ML classifier scores text quality

- Low-quality: spam, SEO content, garbled text

- ◆ Harmful Content Removal

- Violence · Hate speech · Self-harm instructions

- ◆ PII Removal

- Phone numbers, emails, addresses, SSNs

- ◆ Language Filtering

- Define target language mix and proportions

GPT-3: 45 TB raw → 570 GB after filtering (1.3% kept!)

Stage 3 — Tokenization

Converting human text into numbers the model understands

Three possible approaches — only one works well:

Approach 1: Character-level ■ NOT USED

A=1, B=2, C=3... — 'Hello' = [8, 5, 12, 12, 15]

Problem: loses word meaning. 10-letter word = 10 numbers = very long sequences.

Approach 2: Word-level ■ NOT USED

Each word gets a unique ID. English has 170,000+ words.

Problem: 'run', 'running', 'ran' are 3 different entries. Vocabulary is enormous.

Unseen words at inference time become [UNK] (unknown).

Approach 3: Subword Tokenization ■ USED BY ALL LLMs

"tokenization" → ["token", "ization"]

"unhappiness" → ["un", "happi", "ness"]

"GPT-4" → ["G", "PT", "-", "4"]

Balances vocabulary size with linguistic meaning.

Algorithm	Used In	Key Idea
BPE (Byte Pair Encoding)	GPT-2/4, LLaMA	Start with chars, merge most frequent pairs iteratively
WordPiece	BERT, DistilBERT	Like BPE but maximizes likelihood of training data
SentencePiece	LLaMA2, T5, Mistral	Operates on raw bytes — language-agnostic

■ GPT-2: 50,257 tokens | GPT-4: ~100,000 tokens | LLaMA 2: 32,000 tokens

Stage 4 — Pre-training

The most expensive phase — teaching the model language itself

Training Objective: Next Token Prediction

Hardware & Cost Reality:

Training Costs

GPT-3 (175B): ~\$4–12 Million USD

~10,000 NVIDIA A100 GPUs

Several months of training

LLaMA 2 (70B): ~\$2 Million USD

~2,000 A100 GPUs

GPT-2 (117M): ~\$5,000 USD

A few consumer GPUs

Self-Supervised Learning

Input: 'The Eiffel Tower is located in'

Target: 'Paris'

No human labels needed! The next word IS the label.

This is why training can scale to petabytes of data.

Loss Function: Cross-Entropy

Loss = $-\log(P(\text{correct_next_token}))$

If model says 90% chance of correct word → small loss

If model says 2% chance → huge loss → parameters update

■ *Backpropagation adjusts all parameters to minimize loss over billions of examples.*

Chinchilla Scaling Laws (DeepMind, 2022):

- ◆ Optimal training tokens = $20 \times$ parameters
- ◆ 70B model needs ~1.4 Trillion tokens
- ◆ Smaller model + more data > bigger model + less data
- ◆ Most large models were under-trained (too big, too little data)

Quantization — Making Models Fit in Memory

How to run a 70B model on a single GPU

The Memory Problem

1B parameters $\times$ 4 bytes (FP32) = **4 GB** just to load the model

Training at FP32 requires additional ~80 GB VRAM per billion parameters

80 GB = the FULL capacity of a single NVIDIA A100 (most expensive consumer GPU)

A 70B model at FP32 = 280 GB $\rightarrow$ requires 4+ A100s just to load

Solution: Reduce numerical precision

Format	Bits	Memory for 70B model	Trade-off
FP32 (default)	32-bit	280 GB	Full precision, full cost
BF16 / FP16	16-bit	140 GB	Minimal quality loss, 2x faster
INT8	8-bit	70 GB	Slight quality drop, 4x compression
INT4 (QLoRA)	4-bit	35 GB	Noticeable but acceptable for inference

BF16 — Best for Training

Brain Float 16 (Google Brain)

1 sign + 8 exponent + 7 fraction bits

Same dynamic range as FP32

Supported natively by NVIDIA A100+

Gold standard for LLM training

INT4 — Best for Deployment

QLoRA uses 4-bit for base model

π stored as simply 3

Range: -8 to +7

Run LLaMA 70B on a single 48GB GPU

Used in llama.cpp, Ollama, GGUF format

Stage 5 — Fine-Tuning

Transform a text predictor into a useful assistant

The Problem After Pre-training

The model is now an expert at completing text. Ask it 'How do I make a bomb?' and it will complete the sentence naturally — it learned from the internet.

It does NOT refuse. It does NOT help. It predicts the most likely next tokens.

Fine-tuning is what turns this raw model into ChatGPT or Claude.

Supervised Fine-Tuning (SFT):

Feed the model (input, ideal_output) pairs:

Parameter-Efficient Fine-Tuning (PEFT):

LoRA (Low-Rank Adaptation)

Instead of updating ALL weights W (expensive),

we add small matrices: $\Delta W = A \times B$

where rank $r \ll d$ (e.g., $r=4$, $d=4096$)

W : $4096 \times 4096 = 16.7\text{M}$ params

LoRA: $A(4096 \times 4) + B(4 \times 4096) = 32\text{K}$ params

Savings: 99.8% fewer trainable parameters!

```
# SFT Training Example
{
  "input": "Explain gradient descent",
  "output": "Gradient descent is an
optimization algorithm that
minimizes a loss function by
iteratively adjusting parameters..."
}
```

QLoRA

Base model frozen at 4-bit precision

LoRA adapters trained at 16-bit

Fine-tune LLaMA 70B on a single 48GB GPU

◆ Need 1,000+ high-quality human-written examples

- ◆ Usually from expert annotators or crowdsourcing
- ◆ This phase takes hours, not months

Stage 6 — Alignment (RLHF)

Teaching the model to be helpful, harmless, and honest

Problems that remain after SFT:

- ◆ Hallucination — confidently states wrong facts
- ◆ Bias — unfair treatment of different groups
- ◆ Harmful output — dangerous instructions
- ◆ Misunderstood intent — technically correct but unhelpful
- ◆ Verbose/sycophantic — tells you what you want to hear

Solution: RLHF in 3 Steps

Step 1 — SFT (already done)

Model can follow instructions. Now we need to make it prefer GOOD responses.

Step 2 — Train a Reward Model

Give the model the same prompt → generate 4 different responses

Human annotators rank them: Response A > C > B > D

Train a separate Reward Model to predict this ranking

The Reward Model IS the 'human preference' encoded in math

Step 3 — PPO Optimization (Reinforcement Learning)

Main model generates response → Reward Model scores it

PPO algorithm updates the main model to maximize reward

Loop thousands of times → model learns to produce high-reward responses

A 'KL divergence penalty' prevents the model from drifting too far from SFT

RLAIF (AI Feedback)

Replace human annotators with a stronger AI model

100x faster and cheaper than human labeling

Used by Anthropic (Claude grades Claude)

Constitutional AI (Anthropic)

Define a 'constitution' of principles

Model critiques its own responses against it

No humans needed for ranking

Stage 7 — Evaluation & Benchmarks

How do we measure if the model is actually good?

Benchmark	Tests	Size	Why It Matters
MMLU	Knowledge across 57 academic subjects	140,000 questions	Measures breadth of world knowledge
HumanEval	Python code generation	164 problems	Coding ability, algorithmic reasoning
TruthfulQA	Avoidance of confident lies	817 questions	Hallucination resistance
GSM8K	Grade school math word problems	8,500 problems	Multi-step reasoning
HellaSwag	Common sense sentence completion	70,000 samples	Real-world reasoning
ARC-Challenge	Science questions (hard)	1,172 questions	Scientific reasoning
MMLU-Pro	Harder MMLU with reasoning	12,000 questions	Advanced knowledge + reasoning

Perplexity — Technical Metric

Measures how 'surprised' the model is by test text

Lower perplexity = model predicts text better

GPT-2: ~35 perplexity on WikiText

GPT-3: ~20 perplexity on WikiText

Used during training to monitor progress

Critical Warning

Benchmark contamination: if training data contains

benchmark questions, scores are inflated.

Always check multiple benchmarks.

Human evaluation is still the gold standard.

Stage 8 & 9 — Deployment & Monitoring

Getting the model to users reliably and efficiently

Deployment Optimizations:

Quantization (already covered)

FP32 → INT8: 4x memory reduction

FP32 → INT4: 8x memory reduction

Formats: GGUF (CPU), GPTQ (GPU), AWQ

What to Monitor in Production:

◆ Latency: P50, P95, P99 response times

■ Alert if P95 > 2 seconds

KV Cache

Attention computes Key and Value vectors for each token

Cache them so we never recompute past tokens

Saves ~80% computation for long conversations

◆ Hallucination rate via automated fact-checking

◆ User feedback thumbs up/down ratio

◆ Escalation rate (for chatbots)

vLLM — High-Throughput Serving

PagedAttention: manages KV cache like OS memory

Continuous batching: processes requests in parallel

24x faster throughput than naive HuggingFace

◆ Input distribution drift

■ Are prompts changing character?

◆ Token usage and cost per user

Feedback Loop

Collect edge cases that failed

Add them to fine-tuning dataset

Re-train and re-evaluate

Deploy new version — repeat forever

Practical Lab 1 — GPT-2 on Google Colab

5 exercises covering the full lifecycle hands-on

■ Open Colab: colab.research.google.com → New Notebook

Exercise 1: Install & Tokenization

Exercise 1

```
# ■■■■ EXERCISE 1: Tokenization ■■■■

!pip install transformers torch -q
from transformers import GPT2Tokenizer

tokenizer = GPT2Tokenizer.from_pretrained('gpt2')

# See how different words are split
words = ['tokenization', 'unhappiness', 'ChatGPT', '2024', 'AWS', 'Bedrock']
for word in words:
    toks = tokenizer.tokenize(word)
    ids = tokenizer.encode(word)
    print(f'{word:20} -> tokens={toks} ids={ids}')

# Full sentence
sent = 'Large Language Models are revolutionizing AI'
print(f'Sentence: {len(tokenizer.encode(sent))} tokens for {len(sent.split())} words')
print(f'Vocab size: {tokenizer.vocab_size:,}') # should be 50,257
```

Exercise 2: Load Model — Count Parameters

Exercise 2

```
# ■■■■ EXERCISE 2: Model Architecture ■■■■

from transformers import GPT2LMHeadModel

model = GPT2LMHeadModel.from_pretrained('gpt2')
model.eval()

total = sum(p.numel() for p in model.parameters())
print(f'Total parameters: {total:,}') # 124,439,808
print(f'Model size at FP32: {total*4/1e9:.2f} GB')
print(f'Model size at FP16: {total*2/1e9:.2f} GB')
print(f'Model size at INT8: {total*1/1e9:.2f} GB')

# Inspect architecture
print(f'Layers: {model.config.n_layer}') # 12
print(f'Hidden size: {model.config.n_embd}') # 768
print(f'Attention heads: {model.config.n_head}') # 12
```

```
print(f'Context window: {model.config.n_positions}') # 1024
```

Lab 1 — Exercises 3, 4 & 5

Exercise 3: See the Probabilities (Heart of LLM!)

Exercise 3

```
# ■■■■ EXERCISE 3: Next Token Probabilities ■■■■

import torch, torch.nn.functional as F

prompts = [
    'The capital of France is',
    'The most popular programming language is',
    'Machine learning is a subset of',
]

for prompt in prompts:
    inputs = tokenizer(prompt, return_tensors='pt')
    with torch.no_grad():
        logits = model(**inputs).logits[0, -1, :]
        probs = F.softmax(logits, dim=-1)
        top5 = torch.topk(probs, 5)
        print(f'Prompt: "{prompt}"')
        for prob, idx in zip(top5.values, top5.indices):
            word = tokenizer.decode([idx.item()])
            print(f' {word!r:20} {prob.item():.2%}')
        print()
```

Exercise 4: Temperature Effect on Generation

Exercise 4

```
# ■■■■ EXERCISE 4: Temperature ■■■■

from transformers import pipeline, set_seed

gen = pipeline('text-generation', model='gpt2')
prompt = 'Scientists recently discovered that artificial intelligence'

for temp in [0.1, 0.5, 0.9, 1.5]:
    set_seed(42)
    out = gen(prompt, max_new_tokens=40, temperature=temp, do_sample=True)
    print(f'Temp={temp}: {out[0]["generated_text"]}')
    print('-'*70)

# Question: Which temperature gives the most coherent output?
# Question: Which gives the most creative/surprising output?
```

Exercise 5: Mini Fine-Tuning on Custom Data

Exercise 5

```
# ■■■■ EXERCISE 5: Fine-Tuning GPT-2 on AI Facts ■■■■

from transformers import (TextDataset, DataCollatorForLanguageModeling,
Trainer, TrainingArguments)

# Custom training data
AI_FACTS = '''
Transformers use self-attention to process sequences in parallel.
BERT is an encoder-only model trained with masked language modeling.
GPT models are decoder-only and trained with causal language modeling.
Fine-tuning adapts a pre-trained model to a specific downstream task.
RLHF aligns language models with human preferences and values.
LoRA reduces fine-tuning cost by adding low-rank weight matrices.
Quantization reduces model precision to decrease memory requirements.
RAG combines retrieval with generation for knowledge-grounded responses.
'''

with open('/content/ai_facts.txt','w') as f: f.write(AI_FACTS)

tokenizer.pad_token = tokenizer.eos_token
dataset = TextDataset(tokenizer=tokenizer,
file_path='/content/ai_facts.txt', block_size=64)
collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)

args = TrainingArguments('/content/gpt2-ai-facts', num_train_epochs=5,
per_device_train_batch_size=2, logging_steps=5,
prediction_loss_only=True, report_to='none')

Trainer(model=model, args=args,
data_collator=collator, train_dataset=dataset).train()

# Test: does the model now know AI facts?
gen2 = pipeline('text-generation', model=model, tokenizer=tokenizer)
set_seed(42)
print(gen2('Transformers use self-attention to',
max_new_tokens=20)[0]['generated_text'])
```

Hour 2

Decoding Strategies

The technical dials that control creativity, coherence, and reliability

30 min Theory + 30 min Colab Lab (4 exercises + comparison table)

How Does the Model Choose the Next Word?

Understanding the decoding problem from first principles

After the model computes logits...

The model produces a vector of ~50,000 raw scores (logits), one per vocabulary token.

Softmax converts these to probabilities that sum to 100%.

The **decoding strategy** decides how to sample from this distribution.

Example: 'The capital of France is ____'

Paris: 45% | the: 8% | located: 5% | not: 3% | famous: 2% | other ~49k words: 37%

Strategy	Randomness	Quality	Speed	Best Use Case
Greedy Decoding	None (deterministic)	OK	Fastest	Factual Q&A, structured output
Beam Search	None (deterministic)	Good	Slow	Translation, summarization
Temperature Sampling	Controllable	Varies	Fast	General generation
Top-K Sampling	Controllable	Good	Fast	Creative tasks with guardrails
Top-P (Nucleus)	Controllable	Best	Fast	Most production systems

Temperature — The Creativity Dial

The most important parameter for controlling model behavior

The Math

$$P(\text{token}_i) = \text{softmax}(\text{logits} / T)$$

$T \rightarrow 0$: distribution collapses to argmax (always picks highest probability)

$T = 1$: use the raw distribution as-is

$T > 1$: flatten the distribution (rare words get more chance)

Effect on probabilities — 'The capital of France is ____':

Token	Raw Logit	T = 0.1	T = 0.7	T = 1.0	T = 1.5
Paris	8.2	99.3%	78.4%	45.2%	28.7%
the	5.1	0.4%	7.6%	8.1%	9.2%
located	4.8	0.2%	5.1%	4.6%	7.1%
not	4.5	0.1%	3.4%	3.2%	5.8%
known	4.2	0.0%	2.2%	2.1%	4.4%

Low Temperature (0.1 – 0.4)

Use when accuracy matters most

- ◆ Factual Q&A; and fact extraction
- ◆ SQL / code generation
- ◆ Classification tasks
- ◆ Structured data extraction

High Temperature (0.7 – 1.3)

Use when creativity matters most

- ◆ Creative writing, brainstorming
- ◆ Story generation
- ◆ Generating diverse options
- ◆ Role-playing, dialogue

Top-K & Top-P (Nucleus) Sampling

Filtering bad tokens before sampling

Top-K Sampling:

Keep only the top K tokens. Zero out everything else. Sample from remaining K.

Example: K = 5

Paris: 45% ← KEPT

the: 8% ← KEPT

located: 5% ← KEPT

not: 3% ← KEPT

known: 2% ← KEPT

other 49,995 words → EXCLUDED

Probabilities rescaled to sum to 100%.

◆ Weakness: K is fixed regardless of distribution

- If one word has 99% probability, K=50 still picks from 49 bad words
- If all words are equal, K=5 is too narrow

Top-P (Nucleus) Sampling:

Keep fewest words whose cumulative probability $\geq P$. Sample from those.

Example: P = 0.9

Paris: 45% → cumsum: 45%

the: 8% → cumsum: 53%

located: 5% → cumsum: 58%

not: 3% → cumsum: 61%

... continue until cumsum $\geq 90\%$

Dynamic: uses 2 tokens if Paris=89%, or

20 tokens if all probabilities are equal.

◆ Advantage: Adaptive to context

- When model is confident → small nucleus
- When model is uncertain → large nucleus

■ *Production best practice: combine Top-P=0.9 with Temperature=0.7*

Greedy Decoding & Beam Search

Deterministic strategies — no randomness involved

Greedy Decoding:

At every step, pick the single highest-probability token. No exploration.

```
output = model.generate(
    input_ids,
    max_new_tokens=50,
    do_sample=False, # greedy
)
```

- ◆ Pros: Fastest, fully deterministic
- ◆ Cons: Often repetitive and locally optimal
 - 'The cat sat on the mat. The cat sat on the mat.'

■ *Greedy can miss globally good sequences by committing to locally good tokens.*

Beam Search:

Keep top B candidate sequences at every step. Pick best complete sequence at end.

```
output = model.generate(
    input_ids,
    num_beams=5,
    no_repeat_ngram_size=2,
    early_stopping=True,
)
```

- ◆ Pros: Better quality than greedy
- ◆ Cons: 5x slower, memory intensive
- ◆ Cons: Tends to produce safe, boring text
- ◆ Best for: Translation, summarization, structured tasks

■ *Beam Search with n=4 is the standard for machine translation tasks.*

Strategy	do_sample	Other params	When to use
Greedy	False	—	Deterministic code generation, structured extraction
Beam Search	False	num_beams=4-5	Translation, summarization
Temperature	True	temperature=0.7	General chat, creative tasks
Top-K	True	top_k=50, temp=0.7	When vocab is large and diverse
Top-P	True	top_p=0.9, temp=0.7	Most production deployments (default choice)

Inference Parameters — Complete Reference

Know what every parameter does before you tune it

Parameter	Range	What it does	Recommended Default
temperature	0.0 – 2.0	Scales logits before softmax. Controls randomness.	0.7 (chat), 0.1 (factual)
top_p	0.0 – 1.0	Nucleus sampling threshold.	0.9
top_k	1 – 1000	Hard cap on candidate tokens.	50 (or disabled with top_p)
max_new_tokens	1 – 8192+	Hard limit on output length.	256–2048
repetition_penalty	1.0 – 2.0	Penalizes tokens already generated.	1.1 – 1.3
num_beams	1 – 10	Beam search width. 1 = greedy.	1 (chat) 4 (translation)
stop_sequences	list of str	Stop generation when string is found. ['\n\n', 'Human:']	
presence_penalty	0.0 – 2.0	Penalizes any repeated token (OpenAI API).	0.5
frequency_penalty	0.0 – 2.0	Penalizes proportionally to token count.	0.0 – 0.3

- In AWS Bedrock: each model family uses different parameter names. Always check the model's API schema.
- Claude uses: temperature, top_p, top_k, max_tokens, stop_sequences

Lab 2 — Decoding Strategies Comparison

Run all 5 strategies on the same prompt and compare results

■ Continue in the same Colab notebook

Exercise 1: Complete Comparison (run all 5, fill in the table)

```
Exercise 1: All Strategies

# ■■■■ LAB 2: All Decoding Strategies ■■■■

from transformers import GPT2LMHeadModel, GPT2Tokenizer, set_seed
import torch, time

model = GPT2LMHeadModel.from_pretrained('gpt2')
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')
tokenizer.pad_token = tokenizer.eos_token

PROMPT = 'The most important lesson I learned about artificial intelligence is'
inputs = tokenizer(PROMPT, return_tensors='pt')

strategies = {
    'Greedy': dict(do_sample=False),
    'Beam-4': dict(do_sample=False, num_beams=4, no_repeat_ngram_size=2),
    'Temp-0.3': dict(do_sample=True, temperature=0.3),
    'Temp-0.9': dict(do_sample=True, temperature=0.9),
    'Top-P': dict(do_sample=True, top_p=0.9, temperature=0.7),
    'Top-K': dict(do_sample=True, top_k=50, temperature=0.7),
    'Top-P+K': dict(do_sample=True, top_p=0.9, top_k=50, temperature=0.7),
}

print(f'Prompt: "{PROMPT}"\n')
for name, params in strategies.items():
    set_seed(42)
    t0 = time.time()
    out = model.generate(**inputs, max_new_tokens=50,
                        pad_token_id=tokenizer.eos_token_id, **params)
    ms = (time.time()-t0)*1000
    text = tokenizer.decode(out[0], skip_special_tokens=True)
    new = text[len(PROMPT):]
    print(f'[{name:10}] ({ms:5.0f}ms) {new[:120]}')
    print()
```

Exercise 2: Observe Repetition Problem

```
Exercise 2: Repetition

# ■■■■ Exercise 2: Repetition Problem ■■■■
```

```
# Greedy often gets stuck in loops
prompt2 = 'The quick brown fox'
inputs2 = tokenizer(prompt2, return_tensors='pt')

# Without repetition penalty
out = model.generate(**inputs2, max_new_tokens=80, do_sample=False)
print('WITHOUT penalty:', tokenizer.decode(out[0], skip_special_tokens=True))

# With repetition penalty
out = model.generate(**inputs2, max_new_tokens=80, do_sample=False,
repetition_penalty=1.3)
print('WITH penalty: ', tokenizer.decode(out[0], skip_special_tokens=True))
```

Hour 3

AWS Bedrock Ecosystem

Enterprise-grade serverless AI — from Console to Production

30 min Theory + 30 min AWS Console Lab

Why AWS Bedrock?

The economics and engineering case for managed AI

The Problem with Self-Hosting:

- ◆ GPT-4 class model → 100GB+ VRAM required
- ◆ NVIDIA A100 GPU: ~\$10,000 each
- ◆ 8× A100 server: ~\$200,000+ upfront
- ◆ Monthly power: ~\$5,000+
- ◆ DevOps engineer to maintain: \$150K+/year
- ◆ Security compliance = your responsibility
- ◆ Auto-scaling requires significant engineering

What Bedrock Provides:

- ◆ Pay-per-token — zero infrastructure cost
- ◆ 30+ foundation models in one API
- ◆ Enterprise security out of the box
- ◆ Auto-scaling to millions of requests
- ◆ SOC2, HIPAA, GDPR, FedRAMP compliance
- ◆ Your data never trains public models

Bedrock Value Proposition

Focus on building products, not managing GPUs.

Switch models with one line change.

Access Claude, Llama, Titan, Mistral in one SDK.

Realistic Cost

Self-host LLaMA 70B for 1M requests/month: ~\$50K

AWS Bedrock Claude for 1M requests/month: ~\$3-15K

ROI: 60-90% cost saving at medium scale

Bedrock Model Catalog — Choosing the Right Model

Different models for different needs, costs, and constraints

Model	Provider	Strengths	Context	Price/1M tokens
Claude 3.5 Sonnet	Anthropic	Best reasoning, coding, long context	200K	\$3 in / \$15 out
Claude 3 Haiku	Anthropic	Fast, cheap, high-volume chat	200K	\$0.25 in / \$1.25 out
Claude 3 Opus	Anthropic	Maximum intelligence, complex tasks	200K	\$15 in / \$75 out
Llama 3.1 70B	Meta	Open weights, customizable, multilingual	128K	~\$0.99 in / \$0.99 out
Llama 3.1 8B	Meta	Cost-efficient, simple classification	128K	~\$0.22 in / \$0.22 out
Titan Text Premier	Amazon	AWS-native, RAG, enterprise workloads	32K	\$0.50 in / \$1.50 out
Mistral Large	Mistral	European data residency, multilingual	32K	\$4 in / \$12 out
Amazon Nova Pro	Amazon	Multimodal, fast, latest AWS model	300K	\$0.80 in / \$3.20 out

Model Selection Decision Tree

Need max intelligence? → Claude 3.5 Sonnet

Need speed + low cost for chat? → Claude 3 Haiku

Need open weights for fine-tuning? → Llama 3.1 70B

Need European data residency? → Mistral Large

Building RAG on AWS? → Titan Text Premier + Bedrock Knowledge Bases

■ **INSTRUCTOR:** Model Access must be enabled manually. Without this step, NO code will work.

Bedrock Architecture & Security

How your code connects to models — and why it's safe

The Request Flow

Your Application Code

↓ boto3 SDK (Python) / Bedrock SDK

AWS Bedrock API (HTTPS, TLS 1.3)

↓ Inside AWS network (VPC Endpoints available)

Foundation Model (runs in AWS-managed infrastructure)

↓

Response → your application

Security Features

- ◆ VPC Endpoints — traffic never hits public
- ◆ AWS PrivateLink — private connectivity
- ◆ CloudTrail — full audit log of every API call
- ◆ AWS KMS — encrypt data at rest
- ◆ GuardDuty — threat detection
- ◆ No data used for model training (contractual)

Bedrock Guardrails

- ◆ Content filtering (violence, hate speech, adult)
- ◆ PII detection and automatic redaction
- ◆ Topic denial (block specific subjects)
- ◆ Custom word blocklist
- ◆ Grounding check (for RAG — prevent hallucinations)
- ◆ Configurable per deployment — not global

Compliance Certifications

SOC 1, 2, 3 | ISO 27001 | ISO 27017 | ISO 27018

HIPAA Eligible | PCI DSS Level 1 | GDPR Controls | FedRAMP High

Lab 3 — AWS Console Walkthrough

Enable models, explore Playground, compare outputs

■ Go to: console.aws.amazon.com/bedrock | Region: us-east-1

Step 1: Enable Model Access (CRITICAL — do this first!)

1. In the left sidebar: Bedrock configurations → Model access
2. Click 'Modify model access'
3. Enable: Claude 3 Haiku + Titan Text Premier + Llama 3.1 8B
4. Some models require a use-case description (just be honest, 1-2 sentences)
5. Wait 2-5 minutes. Status must show 'Access granted' before any code will work.

Step 2: Use the Playground to Compare Models

1. Go to: Bedrock → Playgrounds → Chat
2. Select Claude 3 Haiku
3. System prompt: 'You are a senior AWS solutions architect. Be concise.'
4. User prompt: 'What are the top 3 benefits of using AWS Bedrock vs self-hosting LLMs?'
5. Note the response, then switch to Titan Text Premier — ask the same question
6. Switch to Llama 3.1 8B — same question again
7. Compare: Which gave the most precise answer? Which was most verbose?

Step 3: Experiment with Inference Parameters

1. Stay in the Playground with Claude 3 Haiku
2. Open 'Inference configuration' on the right side
3. Prompt: 'Write a creative opening line for a sci-fi novel'
4. Run with Temperature=0.1 → note the output
5. Run with Temperature=1.2 → compare: is it more creative?
6. Set Max tokens=20 → what happens?
7. Add stop sequence '.' → what happens?

Step 4: Explore Bedrock Knowledge Bases (Optional Preview)

1. Bedrock → Knowledge Bases → Create knowledge base
2. This is where RAG pipelines live in Bedrock
3. You'll build a full RAG pipeline in Module 4 — just explore the UI today

Hour 4

Hands-on Boto3 Lab

Write production Python code that calls AWS Bedrock — from first call to benchmark script

60 min Intensive Coding Lab (6 progressive exercises)

Setup — IAM Permissions & boto3

Everything you need before writing your first API call

Install & Configure:

```
# Install the AWS SDK
pip install boto3

# Option A: AWS CLI (recommended)
aws configure

# Prompts:
# AWS Access Key ID: AKIA...
# AWS Secret Access Key: xxxxxx
# Default region: us-east-1
# Default output: json

# Option B: Environment variables
import os
os.environ['AWS_ACCESS_KEY_ID'] = 'AKIA...'
os.environ['AWS_SECRET_ACCESS_KEY'] = 'sec...'
os.environ['AWS_DEFAULT_REGION'] = 'us-east-1'
```

Minimum IAM Policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "bedrock:InvokeModel",
        "bedrock:InvokeModelWithResponseStream",
        "bedrock:ListFoundationModels",
        "bedrock:GetFoundationModel"
      ],
      "Resource": "*"
    }
  ]
}
```

■ *Always use least-privilege: only grant the actions your code actually needs.*

Bedrock Client — Key Concepts

Two different clients for two different purposes

bedrock client

Used for management operations:

- ◆ List available models
- ◆ Get model metadata
- ◆ Check model access status
- ◆ NOT for inference

```
client = boto3.client(
    'bedrock',
    region_name='us-east-1'
)
models = client.list_foundation_models()
for m in models['modelSummaries']:
    print(m['modelId'])
```

bedrock-runtime client

Used for inference (actual model calls):

- ◆ `invoke_model` — synchronous, waits for full response
- ◆ `invoke_model_with_response_stream` — streaming
- ◆ `converse` — multi-turn conversation API
- ◆ This is what you use 99% of the time

```
client = boto3.client(
    'bedrock-runtime',
    region_name='us-east-1'
)
# This client does the actual AI work
```

Model ID Format	Example
anthropic.{model-name}-v{N}:{minor}	anthropic.claude-3-haiku-20240307-v1:0
meta.{model-name}-instruct-v{N}:{minor}	meta.llama3-1-8b-instruct-v1:0
amazon.{model-name}-v{N}:{minor}	amazon.titan-text-premier-v1:0
mistral.{model-name}-v{N}:{minor}	mistral.mistral-large-2402-v1:0

Lab 4 — Progressive Boto3 Exercises

6 exercises from first call to production-ready benchmark

■ New Python file: `bedrock_lab.py` (or use Colab)

Exercise 1: Your First Bedrock API Call

Exercise 1

```
# ■■■■ EXERCISE 1: First Invoke ■■■■

import boto3, json

client = boto3.client('bedrock-runtime', region_name='us-east-1')
MODEL_ID = 'anthropic.claude-3-haiku-20240307-v1:0'

body = {
    "anthropic_version": "bedrock-2023-05-31",
    "max_tokens": 256,
    "temperature": 0.7,
    "messages": [{
        "role": "user",
        "content": "What is a Large Language Model? Explain in 3 sentences."
    }]
}

response = client.invoke_model(
    modelId=MODEL_ID,
    body=json.dumps(body),
    contentType='application/json',
    accept='application/json'
)

result = json.loads(response['body'].read())

print('=== RESPONSE ===')
print(result['content'][0]['text'])
print()
print(f'Input tokens: {result["usage"]["input_tokens"]}')
print(f'Output tokens: {result["usage"]["output_tokens"]}')

# Calculate cost (Claude 3 Haiku pricing)
cost = result['usage']['input_tokens'] * 0.00025/1000 + \
result['usage']['output_tokens'] * 0.00125/1000
print(f'This call cost: ${cost:.6f} USD')
```

Lab 4 — Exercises 2, 3 & 4

Exercise 2: System Prompt + Conversation

Exercise 2

```
# ■■■■ EXERCISE 2: System Prompt & Role ■■■■

body = {
    "anthropic_version": "bedrock-2023-05-31",
    "max_tokens": 512,
    "system": "You are an expert AWS Solutions Architect. Always recommend
cost-optimized solutions. Be concise.",
    "messages": [
        {"role": "user", "content": "How should I store ML training data?"},
        {"role": "assistant", "content": "For ML training data, I recommend S3"},
        {"role": "user", "content": "What S3 storage class should I use?"}
    ]
}

response = client.invoke_model(
    modelId='anthropic.claude-3-haiku-20240307-v1:0',
    body=json.dumps(body),
    contentType='application/json', accept='application/json'
)
result = json.loads(response['body'].read())
print(result['content'][0]['text'])

# Q: How does the system prompt change the character of the response?
# Q: Try without system prompt – does the answer change?
```

Exercise 3: Streaming Response

Exercise 3

```
# ■■■■ EXERCISE 3: Streaming ■■■■

body = {
    "anthropic_version": "bedrock-2023-05-31",
    "max_tokens": 1024, "temperature": 0.8,
    "messages": [{"role": "user",
    "content": "Write a short story about an AI that learns to paint."}]
}

response = client.invoke_model_with_response_stream(
    modelId='anthropic.claude-3-haiku-20240307-v1:0',
    body=json.dumps(body),
    contentType='application/json', accept='application/json'
)
```

```
print('=== STREAMING OUTPUT ===', flush=True)
full_text = ''
for event in response['body']:
    chunk = json.loads(event['chunk']['bytes'])
    if chunk['type'] == 'content_block_delta':
        text = chunk['delta'].get('text', '')
        print(text, end='', flush=True) # appear token by token!
        full_text += text
    elif chunk['type'] == 'message_delta':
        tokens = chunk.get('usage', {}).get('output_tokens', 0)
        print(f'\n\n[Output tokens: {tokens}]')
```

Lab 4 — Exercises 4, 5 & 6

Exercise 4: Production-Ready BedrockLLM Class

Exercise 4

```
# ■■■■ EXERCISE 4: Production Client ■■■■

import boto3, json, time
from typing import Optional

class BedrockLLM:
    MODELS = {
        'claude-haiku': 'anthropic.claude-3-haiku-20240307-v1:0',
        'claude-sonnet': 'anthropic.claude-3-5-sonnet-20241022-v2:0',
        'llama-8b': 'meta.llama3-1-8b-instruct-v1:0',
        'titan': 'amazon.titan-text-premier-v1:0',
    }
    COSTS = { # per 1K tokens (input, output)
        'claude-haiku': (0.00025, 0.00125),
        'claude-sonnet': (0.003, 0.015),
        'llama-8b': (0.00022, 0.00022),
        'titan': (0.0005, 0.0015),
    }

    def __init__(self, model_key='claude-haiku', region='us-east-1'):
        self.model_key = model_key
        self.model_id = self.MODELS[model_key]
        self.client = boto3.client('bedrock-runtime', region_name=region)
        self.usage = {'input': 0, 'output': 0, 'calls': 0, 'latency_ms': []}

    def invoke(self, prompt: str, system: Optional[str]=None,
               max_tokens=512, temperature=0.7) -> str:
        messages = [{'role': 'user', 'content': prompt}]
        body = {'anthropic_version': 'bedrock-2023-05-31',
                'max_tokens': max_tokens, 'temperature': temperature,
                'messages': messages}
        if system: body['system'] = system
        t0 = time.time()
        resp = self.client.invoke_model(modelId=self.model_id,
                                         body=json.dumps(body),
                                         contentType='application/json', accept='application/json')
        self.usage['latency_ms'].append((time.time()-t0)*1000)
        result = json.loads(resp['body'].read())
        u = result.get('usage', {})
        self.usage['input'] += u.get('input_tokens', 0)
```

```
self.usage['output'] += u.get('output_tokens',0)
self.usage['calls'] += 1
return result['content'][0]['text']

def stats(self) -> dict:
ci, co = self.COSTS[self.model_key]
cost = self.usage['input']*ci/1000 + self.usage['output']*co/1000
lats = self.usage['latency_ms']
return {'model':self.model_key, 'calls':self.usage['calls'],
'total_tokens':self.usage['input']+self.usage['output'],
'cost_usd':round(cost,6),
'avg_latency_ms':round(sum(lats)/len(lats),1) if lats else 0}
```

Lab 4 — Exercises 5 & 6

Exercise 5: Multi-Model Benchmark

Exercise 5

```
# ■■■■ EXERCISE 5: Multi-Model Benchmark ■■■■

SYSTEM = 'You are a helpful AI assistant. Be concise and precise.'
PROMPT = '''

Write a Python function that:
1. Takes a list of integers
2. Returns the top-3 largest values
3. Handles edge cases: empty list, fewer than 3 items
Include type hints and a docstring.
'''

models = ['claude-haiku', 'llama-8b']
results = {}

for model_key in models:
    llm = BedrockLLM(model_key)
    response = llm.invoke(PROMPT, system=SYSTEM, temperature=0.2)
    results[model_key] = {'response': response, 'stats': llm.stats()}
    print(f'\n=== {model_key.upper()} ===')
    print(response[:600])

# Comparison table
print('\n' + '='*60)
print(f'{'Model':20} {'Tokens':10} {'Latency':12} {'Cost':10}')
for k, v in results.items():
    s = v['stats']
    print(f'{k:20} {s["total_tokens"]:10} {s["avg_latency_ms"]:8.0f}ms
    ${s["cost_usd"]:.6f}')
```

Exercise 6: Token Counting Before Sending (Cost Control)

Exercise 6

```
# ■■■■ EXERCISE 6: Pre-flight Token Count ■■■■

# Count tokens BEFORE calling the API to estimate cost

import boto3
bedrock_mgmt = boto3.client('bedrock', region_name='us-east-1')

# Use Bedrock's token counting endpoint
body = {
    "anthropic_version": "bedrock-2023-05-31",
```

```
"messages": [{"role": "user", "content": PROMPT}]
}

count_resp = client.invoke_model(
    modelId='anthropic.claude-3-haiku-20240307-v1:0',
    body=json.dumps(**body, 'max_tokens':1}), # min tokens to just count
    contentType='application/json', accept='application/json'
)
result = json.loads(count_resp['body'].read())
input_tokens = result['usage']['input_tokens']

# Estimate max cost before sending
max_output = 512
max_cost = input_tokens*0.00025/1000 + max_output*0.00125/1000
print(f'Input tokens: {input_tokens}')
print(f'Max estimated cost: ${max_cost:.6f}')
print(f'Proceed? Run the actual call only if cost is acceptable.')
```

Session Summary — What You Built Today

4 hours, 4 modules, 17 hands-on exercises

Hour	Topic	Key Concepts	Lab Output
1	LLM Lifecycle	Data→Tokenize→Pre-train→SFT→RLHF→Evaluate→Deploy	LLM lifecycle diagram + probabilities + fine-tuning (5 exercises)
2	Decoding Strategies	Greedy/Beam/Temp/Top-K/Top-P + repetition penalty	Strategy comparison on the same prompt
3	AWS Bedrock	Serverless AI + Model Catalog + Security + Guardrails	Consolidated walkthrough + Playground experiments
4	Boto3 Lab	InvokeModel + Streaming + Production-ready Boto3 class	Boto3 Boto class with cost tracking (6 exercises)

Next Session — Module 2: Transformer Architecture

- ◆ Self-Attention mechanism: Query, Key, Value matrices
- ◆ Multi-head attention and why it matters
- ◆ Positional encoding — how models understand sequence order
- ◆ Feed-forward layers and layer normalization
- ◆ Encoder-only (BERT) vs Decoder-only (GPT) vs Encoder-Decoder (T5)

Assignment Before Next Session

1. Extend BedrockLLM class to support Titan Text Premier
2. Add retry logic with exponential backoff for ThrottlingException
3. Compare the cost of answering 1000 questions with Haiku vs Sonnet
4. Read: 'Attention Is All You Need' — Abstract + Section 3 only

References & Further Reading

Foundational Papers:

- ◆ "Attention Is All You Need" — Vaswani et al., Google Brain, 2017 | arxiv.org/abs/1706.03762
- ◆ "Scaling Laws for Neural Language Models" — Kaplan et al., OpenAI, 2020
- ◆ "Training Compute-Optimal LLMs" (Chinchilla) — Hoffmann et al., DeepMind, 2022
- ◆ "LoRA: Low-Rank Adaptation of Large Language Models" — Hu et al., 2022
- ◆ "Training Language Models to Follow Instructions with Human Feedback" — Ouyang et al., 2022
- ◆ "QLoRA: Efficient Finetuning of Quantized LLMs" — Dettmers et al., 2023

AWS Documentation:

- ◆ AWS Bedrock Docs: docs.aws.amazon.com/bedrock
- ◆ Bedrock Pricing: aws.amazon.com/bedrock/pricing
- ◆ boto3 Reference:
boto3.amazonaws.com/v1/documentation/api/latest/reference/services/bedrock-runtime
- ◆ Bedrock Guardrails: docs.aws.amazon.com/bedrock/latest/userguide/guardrails.html

Interactive Learning:

- ◆ LLM Visualization (3D animated): bbycroft.net/llm
- ◆ GenAI with LLMs Notes: github.com/MalayAgr/generative-ai-with-llms-notes
- ◆ Sebastian Raschka LLM Slides: sebastianraschka.com/pdf/slides/2024-build-llms.pdf
- ◆ Hugging Face Course (free): huggingface.co/learn/nlp-course

AWS GenAI Foundation Course | Module 1 | Prepared with Claude